VOLUME 07 OF 10

Computer Engineering and Digital Design

Undergraduate CE core

Move from circuits and gates to RTL, a processor and an FPGA-based game subsystem.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 31 Circuits, measurement and electronic foundations

31.1 Voltage, current, resistance and safe measurement • 31.2 Capacitors, inductors, transients and frequency response • 31.3 Diodes, transistors, CMOS and analog interfaces • 31.4 Sampling, ADCs, DACs and noisy controller inputs

Module 32 Digital logic, state and timing

32.1 Boolean logic and combinational building blocks • 32.2 Registers, finite-state machines and synchronous timing • 32.3 Binary arithmetic, fixed point and overflow policies • 32.4 Clock domains, reset and metastability

Module 33 SystemVerilog and verification-driven RTL design

33.1 RTL semantics and synthesizable SystemVerilog • 33.2 Testbenches, scoreboards and constrained stimulus • 33.3 Assertions, coverage and formal reasoning • 33.4 Synthesis, constraints and FPGA resource mapping

Module 34 Designing a small processor

34.1 Instruction sets, datapaths and single-cycle execution • 34.2 Pipelining, forwarding, stalls and branch recovery • 34.3 Memory interfaces, caches and protection boundaries • 34.4 Processor verification, toolchains and architectural evidence

Module 35 FPGA-based game subsystems and integration

35.1 Memory-mapped I/O, timers, interrupts and DMA • 35.2 Framebuffers, scanout and pixel pipelines • 35.3 A small accelerator and its software contract • 35.4 Integrated digital-design review and optional board bring-up

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 31

Circuits, measurement and electronic foundations

Prerequisites Modules 06-10; basic algebra and calculus competence required. Simulation is the default laboratory.

Four lessons

31.1 Voltage, current, resistance and safe measurement

Analyze a simple circuit using units and conservation laws.

31.2 Capacitors, inductors, transients and frequency response

Relate stored energy to timing and filtering behavior.

31.3 Diodes, transistors, CMOS and analog interfaces

Understand the physical devices beneath digital abstractions.

31.4 Sampling, ADCs, DACs and noisy controller inputs

Convert physical signals into stable digital measurements.

Module project

Create a circuit laboratory notebook using simulation first: voltage/current prediction, an RC transient, a transistor switch and a sampled input. Physical extensions require a qualified mentor and a documented low-voltage safety review.

Core references

R30 MIT OpenCourseWare — 6.002 Circuits and Electronics

R40 OpenStax / Rice University — Precalculus, second edition

R10 MIT OpenCourseWare — 6.003 Signals and Systems

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 31.1 • MODULE 31

Voltage, current, resistance and safe measurement

Learning goal Analyze a simple circuit using units and conservation laws.

Voltage is potential difference between two points; current is charge flow. Resistance relates voltage and current for an ideal resistor through V=IR. Kirchhoff's current law expresses charge conservation at a node, while the voltage law applies around a lumped-circuit loop under its modeling assumptions. Always choose reference directions and keep signs consistent.

Power is P=VI, so a circuit can be logically correct but thermally unsuitable. Real components have ratings and tolerances. A voltmeter is connected across the relevant points; measuring current generally changes the measurement arrangement. Incorrect meter configuration can create a short. Begin in simulation and use only reviewed low-voltage educational hardware for physical work. Do not open mains-powered supplies, displays or battery packs.

WORKED EXAMPLE

An ideal 1,000 ohm resistor across 5 V carries 5 mA and dissipates 25 mW. These are model calculations; a physical build also requires component ratings and safe connections.

PRACTICE + PASS CRITERION

Simulate a two-resistor divider and verify node voltages and current conservation. Pass: predictions include units, the source/load assumptions are stated, and a safety checklist precedes any physical experiment.

ASSIGNED STUDY

R30: lumped-circuit abstraction, resistive networks and basic circuit analysis.

CHECK YOUR UNDERSTANDING

1. What current flows through 1,000 ohms at 5 V?

2. What power is dissipated in that ideal resistor?

3. Why can putting a meter in the wrong current-measurement configuration be dangerous?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 31.2 • MODULE 31

Capacitors, inductors, transients and frequency response

Learning goal Relate stored energy to timing and filtering behavior.

A capacitor stores energy in an electric field and relates current to voltage change through i=C*dv/dt in the ideal model. An inductor relates voltage to current change through v=L*di/dt. These elements make circuit behavior depend on time and initial conditions. An RC network is a useful first-order model for charging, filtering and delay.

For a simple charging RC circuit, the time constant is tau=RC. Frequency-domain analysis uses complex impedance to describe sinusoidal steady-state behavior, but it does not replace startup analysis. Real parasitic resistance, inductance and capacitance matter increasingly as signal edges become fast. A model should state which nonideal effects it intentionally omits.

WORKED EXAMPLE

With R=10,000 ohms and C=1 microfarad, tau=0.01 s, or 10 ms. Starting from zero, the ideal capacitor reaches about 63.2% of its final step-response voltage after one time constant.

PRACTICE + PASS CRITERION

Simulate an RC step response and a sinusoidal frequency sweep. Pass: the measured time constant agrees with the ideal calculation within numerical tolerance, and the report distinguishes transient from steady-state behavior.

ASSIGNED STUDY

R30: capacitors, inductors, first- and second-order systems; R10: frequency response.

CHECK YOUR UNDERSTANDING

1. What is tau for 10 kohm and 1 microfarad?

2. What fraction of a zero-to-final RC charging step is reached at one tau?

3. Why do initial conditions matter?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 31.3 • MODULE 31

Diodes, transistors, CMOS and analog interfaces

Learning goal Understand the physical devices beneath digital abstractions.

A diode has a nonlinear current-voltage relationship; it is not a perfect one-way switch. A transistor can control current and serve as an amplifier or switch depending on its operating region and circuit. MOS devices form complementary networks in CMOS logic. The digital 0/1 abstraction hides voltage ranges, delay, leakage and transition behavior.

Analog interfaces may use amplifiers, comparators and reference voltages. An ideal operational-amplifier model is useful only while feedback and operating limits support its assumptions. Real inputs and outputs have range, bandwidth and loading constraints. Read datasheets as contracts, including absolute maximum ratings and recommended operation, rather than choosing parts from one headline specification.

WORKED EXAMPLE

A logic input specified for one voltage domain is not automatically safe when driven by a higher-voltage output. A logical high in both systems does not imply electrical compatibility.

PRACTICE + PASS CRITERION

Simulate a CMOS inverter and one simple amplifier or comparator stage. Pass: plot input/output behavior, identify valid operating regions and explain one failure of the idealized model.

ASSIGNED STUDY

R30: MOS transistors, amplifiers, digital circuits and operational amplifiers.

CHECK YOUR UNDERSTANDING

1. Why is a transistor not only a binary software switch?

2. Does a logical high guarantee compatibility across voltage domains?

3. Why distinguish absolute maximum ratings from recommended operating conditions?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 31.4 • MODULE 31

Sampling, ADCs, DACs and noisy controller inputs

Learning goal Convert physical signals into stable digital measurements.

An analog-to-digital converter samples and quantizes an input relative to a reference. Resolution describes code granularity, not guaranteed accuracy. Noise, offset, reference error, nonlinearity and acquisition behavior can dominate the nominal bit count. A digital-to-analog converter performs a related reconstruction task with its own filtering and output limits.

An analog signal needs bandwidth control before sampling when aliasing is a concern. Digital filtering afterward cannot generally undo already aliased information. A controller input also needs calibration, dead zones and careful response tuning. Switch debouncing and analog filtering solve different problems. Preserve enough timing information to measure the latency added by filtering.

WORKED EXAMPLE

An ideal 12-bit converter spanning 0 to 3.3 V has a nominal step of about 3.3/4096 = 0.806 mV under that quantization convention. Real usable precision may be lower.

PRACTICE + PASS CRITERION

Simulate a noisy joystick signal, quantization and a simple filter. Pass: report step size, calibration error, added response delay and behavior near the dead-zone boundary.

ASSIGNED STUDY

R30: mixed-signal and measurement concepts; R10: sampling and filtering.

CHECK YOUR UNDERSTANDING

1. How many distinct codes does a 12-bit converter provide?

2. Does 12-bit resolution guarantee 12-bit accuracy?

3. Why filter before sampling to prevent aliasing?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 32

Digital logic, state and timing

Prerequisites Modules 06, 15-16 and 31.

Four lessons

32.1 Boolean logic and combinational building blocks

Translate a logical specification into a circuit with no stored state.

32.2 Registers, finite-state machines and synchronous timing

Model state transitions at clock boundaries.

32.3 Binary arithmetic, fixed point and overflow policies

Choose representations that preserve the needed range and precision.

32.4 Clock domains, reset and metastability

Recognize where synchronous assumptions stop applying.

Module project

Build and test an arithmetic unit, a finite-state controller and a synchronous queue model. For every block, provide a truth table or transition specification before coding its implementation.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R31 Noam Nisan and Shimon Schocken — Nand to Tetris

R35 RISC-V International — RISC-V ISA specifications

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 32.1 • MODULE 32

Boolean logic and combinational building blocks

Learning goal Translate a logical specification into a circuit with no stored state.

Combinational logic computes outputs from current inputs. Boolean algebra and truth tables specify behavior independently of a particular gate arrangement. Multiplexers select among inputs; decoders map a code to selected outputs; adders combine bit patterns according to arithmetic rules. Equivalent logic can have different area, delay and power.

Minimization is not only reducing the printed expression. Fanout, gate availability and signal arrival times influence implementation. A combinational loop usually indicates a design error in beginner synchronous work. Unknown or invalid input codes require a defined policy. Verify small blocks exhaustively before composing them into larger systems.

WORKED EXAMPLE

A two-input AND function has output 1 only for input pair 11. A two-to-one multiplexer computes output=a when select=0 and output=b when select=1.

PRACTICE + PASS CRITERION

Implement a small multiplexer, decoder and four-bit adder in a logic simulator or model. Pass: exhaustive truth-table tests cover every supported input and no state is required for a result.

ASSIGNED STUDY

R15: combinational logic; R31: Boolean logic and arithmetic projects.

CHECK YOUR UNDERSTANDING

1. What distinguishes combinational from sequential logic?

2. What does a two-to-one multiplexer do?

3. Does the shortest Boolean expression always imply the fastest circuit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 32.2 • MODULE 32

Registers, finite-state machines and synchronous timing

Learning goal Model state transitions at clock boundaries.

A register stores state across clock events. A synchronous design computes next-state values from current state and inputs, then captures them at the active clock edge. A finite-state machine makes this transition function explicit. Moore-style outputs depend on state; Mealy-style outputs can also depend directly on inputs, with timing consequences.

Combinational paths between registers must settle within the available timing budget, including clock-to-output, logic, setup and clock uncertainty. Hold requirements constrain how soon data may change after capture. Slowing a clock does not generally fix hold violations. Define reset state and behavior for illegal states rather than relying on power-up luck.

WORKED EXAMPLE

If clock-to-output is 1 ns, logic delay is 6 ns and setup is 1 ns, a simplified zero-skew setup budget needs at least 8 ns per cycle before additional margin, corresponding to at most 125 MHz.

PRACTICE + PASS CRITERION

Design a controller with idle, running and done states, including reset and abort. Pass: a transition table matches simulation and a timing worksheet includes setup and hold as different checks.

ASSIGNED STUDY

R15: sequential logic and finite-state machines; R31: sequential logic.

CHECK YOUR UNDERSTANDING

1. What does a register capture in a synchronous model?

2. What simplified maximum frequency follows from an 8 ns period?

3. Does slowing the clock automatically fix hold violations?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 32.3 • MODULE 32

Binary arithmetic, fixed point and overflow policies

Learning goal Choose representations that preserve the needed range and precision.

Unsigned n-bit arithmetic represents values from zero through 2^n-1. Two's-complement signed arithmetic represents -2^(n-1) through 2^(n-1)-1. Hardware addition can use similar structures for both, but overflow interpretation differs. Explicitly decide whether results wrap, saturate or signal an error.

Fixed-point representations assign an implicit scale to integers. Multiplication changes the scale and usually needs a wider intermediate result. Rounding and truncation introduce error; repeated biased truncation can accumulate. Hardware width affects area and timing as well as numerical behavior. Compare a fixed-point implementation with a higher-precision reference over the complete supported input range where feasible.

WORKED EXAMPLE

A signed 8-bit two's-complement value ranges from -128 to 127. In a format representing integer_value/16, stored integer 24 represents 1.5, and multiplication requires rescaling the product.

PRACTICE + PASS CRITERION

Implement wrapping and saturating addition plus a fixed-point multiply. Pass: boundary cases are exhaustive for a small width and an error report quantifies rounding behavior against a floating-point reference.

ASSIGNED STUDY

R15: arithmetic; R31: ALU construction; R35: integer arithmetic semantics as a software/hardware contract.

CHECK YOUR UNDERSTANDING

1. What is the signed 8-bit two's-complement range?

2. What does stored value 24 mean with scale 1/16?

3. Why use a wider intermediate for multiplication?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 32.4 • MODULE 32

Clock domains, reset and metastability

Learning goal Recognize where synchronous assumptions stop applying.

An asynchronous input can change near a sampling edge and violate a register's timing requirements, allowing metastable behavior. A synchronizer reduces the probability that instability propagates, but does not make the probability mathematically zero. Digital functional simulation generally does not reproduce analog metastability faithfully.

Single-bit control crossings, pulses and multi-bit data need different protocols. Synchronizing every bit of a changing bus independently does not guarantee a coherent word. Handshakes or asynchronous FIFOs transfer data with explicit ownership and timing assumptions. Reset assertion and deassertion also need a domain-aware policy. A clock-domain-crossing review is a design activity, not only a waveform inspection.

WORKED EXAMPLE

A binary counter changing from 0111 to 1000 flips several bits. Independently sampled bits can briefly form a word that was never the intended source value.

PRACTICE + PASS CRITERION

Model a request/acknowledge crossing and explain its stability assumptions. Pass: the receiver never accepts a partially updated word in the protocol model, and the report states that simulation is not an analog metastability proof.

ASSIGNED STUDY

R15: synchronization and asynchronous inputs; deepen with R50 in later verification lessons.

CHECK YOUR UNDERSTANDING

1. Does a two-register synchronizer eliminate all metastability risk?

2. Why not independently synchronize every bit of a changing binary bus?

3. Can ordinary digital simulation prove an analog metastability failure rate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 33

SystemVerilog and verification-driven RTL design

Prerequisites Module 32; Python and C++ foundations complete.

Four lessons

33.1 RTL semantics and synthesizable SystemVerilog

Write a hardware description rather than a sequential software imitation.

33.2 Testbenches, scoreboards and constrained stimulus

Check behavior against an independent specification.

33.3 Assertions, coverage and formal reasoning

Distinguish observed tests from properties proven under assumptions.

33.4 Synthesis, constraints and FPGA resource mapping

Connect verified behavior to a realizable implementation.

Module project

Implement a parameterized synchronous FIFO in RTL with a Python or C++ reference model, automated tests, assertions and synthesis reports. Keep testbench behavior separate from synthesizable hardware.

Core references

R32 Verilator project — Verilator user guide

R33 YosysHQ — Yosys documentation

R34 cocotb project — cocotb documentation

R50 YosysHQ — SymbiYosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 33.1 • MODULE 33

RTL semantics and synthesizable SystemVerilog

Learning goal Write a hardware description rather than a sequential software imitation.

Register-transfer-level code describes concurrent hardware blocks and state updates. Combinational processes should assign every output on every path unless a latch is intentionally specified. Clocked processes describe stored state. Blocking and nonblocking assignments have simulation scheduling semantics; a common disciplined style uses blocking assignments for combinational calculations and nonblocking assignments for clocked state.

Not every language construct is synthesizable, and supported subsets vary by tool. Delays useful in a testbench are not a portable way to create hardware timing. Width, signedness and implicit extension need explicit review. A loop with a fixed elaboration bound may replicate hardware rather than run sequentially over clock cycles. Read synthesis output to check what was actually inferred.

WORKED EXAMPLE

In a clocked block, a<=b; b<=a; with nonblocking assignments exchanges the old register values at the update event. Sequential software assignment with no temporary would behave differently.

PRACTICE + PASS CRITERION

Implement a counter and two-register swap using a simulator. Pass: tests explain old/new values at each edge, lint warnings are resolved or justified, and no unintended latches are inferred.

ASSIGNED STUDY

R32: language support, warnings and simulation; R33: synthesis concepts and supported constructs.

CHECK YOUR UNDERSTANDING

1. Why assign every combinational output on every path?

2. What does the nonblocking swap example do?

3. Does a source-code loop necessarily take multiple hardware clock cycles?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 33.2 • MODULE 33

Testbenches, scoreboards and constrained stimulus

Learning goal Check behavior against an independent specification.

A testbench drives inputs and observes outputs under a timing protocol. A scoreboard compares accepted transactions with a reference model. The reference should express the specification more simply than the RTL, not copy its internal implementation bug for bug. Directed tests target known boundaries; randomized tests explore combinations within a controlled legal domain.

Record random seeds and failing transaction traces so failures are reproducible. Backpressure, reset during activity and simultaneous operations are important for queues and streaming blocks. Sampling before outputs settle can make a correct design appear wrong; testbench scheduling must match simulator semantics. Verification code deserves review and tests of its own.

WORKED EXAMPLE

A FIFO scoreboard enqueues only when a write handshake is accepted and dequeues only when a read handshake is accepted. Counting attempted writes during full conditions would incorrectly blame the device for dropped data.

PRACTICE + PASS CRITERION

Create a FIFO testbench with empty, full, simultaneous read/write and reset cases. Pass: an intentionally corrupted RTL version is detected, and a seed plus trace reproduces the first mismatch.

ASSIGNED STUDY

R34: writing testbenches, triggers and simulator timing; R32: simulation and tracing.

CHECK YOUR UNDERSTANDING

1. Should a scoreboard count attempted or accepted transactions?

2. Why save random seeds?

3. Why deliberately inject a known bug?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 33.3 • MODULE 33

Assertions, coverage and formal reasoning

Learning goal Distinguish observed tests from properties proven under assumptions.

An assertion states a property that should hold, such as occupancy remaining within bounds. Temporal properties describe relationships over time, including data stability while stalled or eventual completion under assumptions. Coverage asks whether important scenarios occurred or are reachable; it is not the same as correctness.

Formal tools explore a mathematical model under explicit assumptions. Bounded checking proves only the explored bound unless stronger reasoning establishes an unbounded result. Overconstraining inputs can make a broken design pass vacuously. Include cover goals and review assumptions separately from assertions. A proof of one interface property is not proof of the whole chip, toolchain or physical implementation.

WORKED EXAMPLE

If a FIFO depth is 8, assert 0<=occupancy<=8. A separate cover goal should demonstrate reaching full and later returning to empty under legal traffic, so the environment is not constrained to do nothing.

PRACTICE + PASS CRITERION

Add occupancy, ordering and stalled-data assertions. Pass: bounded results state their depth, at least two meaningful cover goals are reached, and each environment assumption is justified.

ASSIGNED STUDY

R50: getting started, FIFO example, safety, cover and liveness; R34: assertion-oriented simulation checks.

CHECK YOUR UNDERSTANDING

1. Does reaching 100% of chosen coverage bins prove correctness?

2. Why can overconstrained assumptions hide a bug?

3. What limitation must accompany a bounded proof?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 33.4 • MODULE 33

Synthesis, constraints and FPGA resource mapping

Learning goal Connect verified behavior to a realizable implementation.

Synthesis transforms a supported hardware description into a logic network mapped to available resources. FPGA resources include lookup tables, registers, memories and specialized arithmetic blocks, depending on the device. Coding style can influence whether a memory or multiplier is inferred efficiently, but reports must confirm the result.

Timing constraints tell implementation tools which clocks and paths matter. Missing or incorrect constraints can make a timing report misleading. Functional simulation, synthesis success and timing closure are separate milestones. Compare equivalent implementations using the same device, constraints and tool settings. A small resource count is not useful if the design fails its target timing or interface behavior.

WORKED EXAMPLE

A queue described as individual registers may use a very different amount of FPGA fabric than one mapped to a supported memory primitive. Both can be functionally correct but differ in ports and read latency.

PRACTICE + PASS CRITERION

Synthesize the FIFO and compare two storage implementations or depths. Pass: record device/model, tool version, inferred memory, registers and timing constraints; reconcile any post-synthesis behavior differences.

ASSIGNED STUDY

R33: synthesis and memory mapping; R32: lint; selected FPGA tool documentation only after a target is chosen.

CHECK YOUR UNDERSTANDING

1. Does successful synthesis establish timing closure?

2. Why inspect inferred memory resources?

3. Why use identical constraints when comparing designs?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 34

Designing a small processor

Prerequisites Modules 16 and 32-33. Implement a documented subset, not an unqualified full-ISA claim.

Four lessons

34.1 Instruction sets, datapaths and single-cycle execution

Separate programmer-visible behavior from its implementation.

34.2 Pipelining, forwarding, stalls and branch recovery

Preserve architectural behavior while overlapping instructions.

34.3 Memory interfaces, caches and protection boundaries

Design memory behavior as a protocol rather than an instantaneous array.

34.4 Processor verification, toolchains and architectural evidence

Demonstrate instruction correctness with reproducible programs and traces.

Module project

Create a simple processor model and RTL implementation for a clearly specified instruction subset. Run the same short programs on an instruction-level reference and the RTL, comparing architectural state at retirement.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R31 Noam Nisan and Shimon Schocken — Nand to Tetris

R35 RISC-V International — RISC-V ISA specifications

R32 Verilator project — Verilator user guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 34.1 • MODULE 34

Instruction sets, datapaths and single-cycle execution

Learning goal Separate programmer-visible behavior from its implementation.

An instruction set defines observable operations, encodings, registers and behavior. A microarchitecture implements that contract through a datapath and control. A simple processor may fetch, decode, read operands, execute and update state in one conceptual cycle. Its clock period is limited by the longest active path, so simplicity of control can cost frequency.

Choose a small supported subset and document unsupported instructions, alignment and exception behavior. A processor executing a few RISC-V-like operations is not automatically a compliant RISC-V implementation. Keep the reference interpreter independent from the RTL. Instruction tests should inspect final and intermediate architectural state rather than only whether a display changes.

WORKED EXAMPLE

An ADD instruction might read registers x2 and x3, add their bit patterns under the ISA rule and write x1. The ISA specifies that result; it does not require a particular adder topology.

PRACTICE + PASS CRITERION

Implement arithmetic, load/store and one branch form in a reference model and datapath. Pass: hand-encoded programs produce the expected register and memory state and unsupported encodings have defined behavior.

ASSIGNED STUDY

R35: selected base integer instructions; R15: processor design; R31: computer architecture projects.

CHECK YOUR UNDERSTANDING

1. How does an ISA differ from a microarchitecture?

2. What limits a simple single-cycle clock period?

3. Why avoid claiming full ISA compliance for a small subset?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 34.2 • MODULE 34

Pipelining, forwarding, stalls and branch recovery

Learning goal Preserve architectural behavior while overlapping instructions.

Pipelining overlaps stages from different instructions to improve throughput. It does not necessarily reduce the latency of one instruction. Data hazards arise when a consumer needs a result that is not yet available; forwarding and stalls resolve them under precise timing rules. Structural hazards occur when instructions compete for a resource.

Control hazards follow branches and other changes in instruction flow. A simple design can stall until the branch is known, while more advanced designs predict and recover. Flushed instructions must not create architectural side effects. Track valid bits and distinguish a bubble from a real instruction encoding. Compare retirement behavior with the reference model after every accepted instruction.

WORKED EXAMPLE

If instruction B uses a register produced by A immediately before it, B may need a forwarded result or a stall. Reading the old register-file value produces a data-dependent failure.

PRACTICE + PASS CRITERION

Pipeline the processor or model a pipeline with explicit valid bits. Pass: dependent arithmetic, load-use, branch-taken and back-to-back memory tests match the architectural reference.

ASSIGNED STUDY

R15: pipelined processors; R35: instruction semantics; R32: tracing and debugging.

CHECK YOUR UNDERSTANDING

1. Does pipelining always shorten one instruction's latency?

2. Why must flushed instructions suppress writes?

3. What is a load-use hazard?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 34.3 • MODULE 34

Memory interfaces, caches and protection boundaries

Learning goal Design memory behavior as a protocol rather than an instantaneous array.

A real memory interface can accept requests and return responses later, with stalls or errors. The processor must preserve request identity and avoid repeating side effects while waiting. Separate instruction/data ports simplify some designs but still require a stated memory model. Memory-mapped device registers may have effects that ordinary RAM does not.

A cache holds copies of memory blocks and needs tag comparison, replacement and write policies. Dirty data must reach the appropriate backing store according to the policy. Address translation and protection are additional functions, not inherent properties of every small CPU. An educational core may deliberately omit an MMU while documenting its consequences and deployment limits.

WORKED EXAMPLE

A store request held high during a multi-cycle stall must not become three separate writes if the interface accepts it only once. A ready/valid handshake defines the acceptance event.

PRACTICE + PASS CRITERION

Add a variable-latency memory model and an optional direct-mapped cache. Pass: randomized stalls preserve program results, each accepted store occurs exactly as specified and cache tests include conflicting addresses.

ASSIGNED STUDY

R15: caches, memory and I/O; R35: memory access and selected privilege concepts as an extension.

CHECK YOUR UNDERSTANDING

1. Why is an instantaneous array an incomplete memory model?

2. What does a cache tag identify?

3. Does a small CPU necessarily need an MMU?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 34.4 • MODULE 34

Processor verification, toolchains and architectural evidence

Learning goal Demonstrate instruction correctness with reproducible programs and traces.

Processor testing combines directed instruction cases, randomized legal programs, reference-model comparison and interface assertions. Compare architectural state at defined retirement points, not every internal signal against a different microarchitecture. A useful failure report includes the program, initial state, seed and first mismatching retirement.

A software toolchain adds assembler, linker layout, startup code and calling-convention assumptions. A C compiler may emit instructions outside a tiny core's subset unless configured and checked. Inspect generated disassembly. Formal or compliance suites can strengthen evidence, but their scope and configuration must be recorded. Passing selected tests is not a universal correctness certificate.

WORKED EXAMPLE

A compiled division operation may call a runtime helper or emit an instruction the core does not support. The executable must be inspected and linked against compatible support code.

PRACTICE + PASS CRITERION

Run at least twenty directed programs and a reproducible randomized set against the reference. Pass: mismatch localization works, binary layout is documented and a supported compiled or assembled program runs end to end.

ASSIGNED STUDY

R35: exact chosen ISA revision; R32: simulation; R15: software/processor interface.

CHECK YOUR UNDERSTANDING

1. Why compare at retirement rather than arbitrary internal cycles?

2. Why inspect compiler-generated instructions?

3. What belongs in a reproducible processor failure report?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 35

FPGA-based game subsystems and integration

Prerequisites Modules 21, 31-34. Physical deployment is optional; simulated I/O satisfies the core project.

Four lessons

35.1 Memory-mapped I/O, timers, interrupts and DMA

Explain how software controls asynchronous hardware.

35.2 Framebuffers, scanout and pixel pipelines

Produce timed pixel data with correct storage and scheduling.

35.3 A small accelerator and its software contract

Measure whether specialized hardware improves a complete operation.

35.4 Integrated digital-design review and optional board bring-up

Close the loop between specification, verification and implementation.

Module project

Gate G: demonstrate a verified RTL game subsystem, such as a sprite compositor, controller interface or bounded vector unit. Include a software reference, interface specification, automated verification and synthesis evidence. A physical board is an extension, not a prerequisite.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R32 Verilator project — Verilator user guide

R33 YosysHQ — Yosys documentation

R34 cocotb project — cocotb documentation

R35 RISC-V International — RISC-V ISA specifications

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 35.1 • MODULE 35

Memory-mapped I/O, timers, interrupts and DMA

Learning goal Explain how software controls asynchronous hardware.

Memory-mapped I/O exposes device control through addressable registers with defined access widths and side effects. Reads can acknowledge status or pop data; writes can launch work. Ordinary RAM assumptions may therefore be unsafe. Language volatile alone does not provide inter-thread synchronization or every device-ordering guarantee; use the platform's specified access and barrier mechanisms.

Timers and interrupts notify software of events, while direct memory access moves data without having the CPU copy every element. DMA still needs valid buffers, ownership, addressing and cache-coherence handling. An interrupt handler should perform bounded urgent work and defer expensive processing. A device that can stall or fail requires timeout and recovery policy.

WORKED EXAMPLE

A DMA engine reading a buffer cannot safely use it after software frees or repurposes it. Completion must transfer ownership back before reuse.

PRACTICE + PASS CRITERION

Implement a simulated timer and memory-mapped control block. Pass: register side effects are specified, interrupts are acknowledged without losing new events and buffers remain owned until completion.

ASSIGNED STUDY

R15: I/O and interrupts; R35: ordering concepts for the chosen platform; R32: device testbench modeling.

CHECK YOUR UNDERSTANDING

1. Why are device registers not ordinary memory variables?

2. When may software reuse a DMA source buffer?

3. Does C++ volatile make a shared device driver thread-safe?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 35.2 • MODULE 35

Framebuffers, scanout and pixel pipelines

Learning goal Produce timed pixel data with correct storage and scheduling.

A framebuffer stores pixels, while scanout emits them according to a display timing contract. Active pixels may occupy only part of a complete line/frame interval; blanking and synchronization are additional timing responsibilities. Use documented board/display interfaces rather than inventing electrical signaling. A simulator can capture emitted pixels into image files without any display hardware.

Pixel pipelines must deliver data at the required rate, including memory latency. Double buffering separates the image being displayed from the one being written, but swapping needs a safe synchronization point to avoid tearing. Resolution, color depth and refresh rate determine storage and minimum active-data bandwidth. Real interfaces add overhead and may require more bandwidth.

WORKED EXAMPLE

A 320 x 240 framebuffer at 16 bits per pixel needs 153,600 bytes. Reading only active pixels at 60 frames per second requires 9.216 MB/s, excluding blanking and other traffic.

PRACTICE + PASS CRITERION

Create a simulated scanout counter and color-bar/sprite generator. Pass: exact active dimensions and frame boundaries are verified, captured images match the reference and buffer swapping occurs only at the declared boundary.

ASSIGNED STUDY

R15: digital-system timing; R32 and R34: timed testbenches; use the selected board vendor's documented video reference only for a physical extension.

CHECK YOUR UNDERSTANDING

1. How many bytes does the example framebuffer require?

2. Is active-pixel bandwidth the whole physical interface bandwidth?

3. Why synchronize framebuffer swaps?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 35.3 • MODULE 35

A small accelerator and its software contract

Learning goal Measure whether specialized hardware improves a complete operation.

An accelerator implements a selected computation in dedicated hardware. Its value depends on data reuse, parallelism, setup, transfer and synchronization costs. A tiny arithmetic kernel may run quickly in isolation yet lose to the CPU after communication. Choose a bounded operation with a clear reference, such as vector addition, sprite blending or a fixed-size filter.

Specify input format, address limits, alignment, completion, errors and cancellation. Fixed-point arithmetic must include rounding and saturation rules. Backpressure lets downstream capacity limit upstream production without dropping data. Measure latency to first result, steady-state throughput and total end-to-end time separately. A faster clock is only one possible improvement.

WORKED EXAMPLE

If setup and transfer take 80 microseconds and computation takes 20, the operation takes 100 microseconds. Halving computation alone reduces it to 90, not 50.

PRACTICE + PASS CRITERION

Build a streaming arithmetic or pixel block plus a software driver model. Pass: outputs match the reference over boundary cases, backpressure preserves order and end-to-end cost includes setup and transfer.

ASSIGNED STUDY

R33: synthesis and arithmetic mapping; R34: scoreboard verification; R15: parallelism and I/O.

CHECK YOUR UNDERSTANDING

1. What total time follows from halving the example 20 microsecond compute phase?

2. What does backpressure accomplish?

3. Why report latency and throughput separately?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 35.4 • MODULE 35

Integrated digital-design review and optional board bring-up

Learning goal Close the loop between specification, verification and implementation.

Integration checks clocks, resets, widths, address maps and ownership across blocks. A design can pass every isolated unit test and still fail because two units disagree about a handshake or reset sequence. Maintain an interface table and test the complete path with randomized stalls and reset events.

For a physical extension, use a documented educational board, approved power source and reviewed pin constraints. Verify voltage standards before connecting peripherals. Begin with a known-good reference and add one subsystem at a time. Simulation and FPGA operation provide different evidence; neither validates an ASIC fabrication flow by itself. The final report should clearly label simulated, synthesized and physically measured results.

WORKED EXAMPLE

A functional model may assume memory responds in one cycle while an integrated controller takes several. An end-to-end randomized-latency test exposes that assumption before hardware deployment.

PRACTICE + PASS CRITERION

Complete Gate G with an integration test, synthesis report and short demonstration. Pass: another person reruns the simulation, all timing assumptions are listed and any physical measurements are distinguished from modeled estimates.

ASSIGNED STUDY

R32-R34: repeatable simulation and synthesis; R15: system integration; handbook safety review for physical extensions.

CHECK YOUR UNDERSTANDING

1. Why are unit tests insufficient for integration?

2. What must be checked before wiring a peripheral?

3. Does a successful FPGA demonstration prove an ASIC is ready to fabricate?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.